УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«МОГИЛЕВСКИЙ ГОСУДАРСТВЕННЫЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

Специальность 5-04-0612-02

Учебный предмет Инструментальное программное обеспечение

УТВЕРЖДАЮ
Заместитель директора
по учебной работе
_____М.М.Федоськова
_____

ЛАБОРАТОРНАЯ РАБОТА № 5

СОЗДАНИЕ ПРОСТЕЙШЕЙ ПРОГРАММЫ PYTHON ДЛЯ АВТОМАТИЗИРОВАННОЙ ОБРАБОТКИ ЕСТЕСТВЕННОГО ЯЗЫКА

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

Разработал преподаватель
Денисовец Д.А.

2026

Обсуждено и одобрено
на заседании цикловой комиссии
специальностей в области
программного обеспечения
информационных систем
Протокол № ____от _____
Председатель цикловой комиссии
_____ Д.А.Денисовец

1 Цель работы

1.1 Создание простейшей программы Python для автоматизированной обработки естественного языка

2 Методическое и материальное обеспечение

2.1 Персональный компьютер
2.2 Методические рекомендации по выполнению лабораторной работы
2.3 Интегрированная среда разработки PyCharm / Visual Studio Code

3 Последовательность выполнения работы

3.1 Ознакомиться с основными теоретическими положениями
3.2 Выполнить индивидуальное задание
3.3 Оформить отчет
3.4 Ответить на контрольные вопросы

4 Основные теоретические положения

4.1 Строки в Python

Строки представляют собой упорядоченные последовательности символов. Длина строки ограничена лишь объемом оперативной памяти компьютера. Как и все последовательности, строки поддерживают обращение к элементу по индексу, получение среза, конкатенацию (оператор +), повторение (оператор *), проверку на вхождение (операторы in и not in).

Кроме того, строки относятся к неизменяемым типам данных. Поэтому практически все строковые методы в качестве значения возвращают новую строку. При использовании небольших строк это не приводит к каким-либо проблемам, но при работе с большими строками можно столкнуться с проблемой нехватки памяти. Иными словами, можно получить символ по индексу, но изменить его будет нельзя.

Попытка изменить символ по индексу:

```
s= "Python"
print(s[0]) # Можно получить символ по индексу
```

Результат:

```
P
print([0] = "J") # Изменить строку нельзя
Traceback (most recent call last):
   File "<pyshell#2>", line l, in <module>
      s[0] = "J" # Изменить строку нельзя
TypeError: 'str' object does not support item assignment
```

В некоторых языках программирования концом строки является нулевой символ. В языке Python нулевой символ может быть расположен внутри строки:

```
"string\x00string" # Нулевой символ – Это НЕ конец строки
```

Результат:

'string\x00string'

Язык Python 3 поддерживает следующие строковые типы:

– str – Unicode-строка. Обратите внимание, конкретная кодировка: UTF-8, UTF-16 или UTF-32 – здесь не указана. Рассматривайте такие строки, как строки в некой абстрактной кодировке, позволяющие хранить символы Unicode и производить манипуляции с ними. При выводе Unicode-строку необходимо преобразовать в последовательность байтов в какой-либо кодировке:

```
type ("строка")
<class 'str'>
"строка".encode (encoding="cp1251")
b'\xf1\xf2\xf0\xee\xea\xe0'
"строка".encode (encoding="utf-8")
b'\xd1\x81\xd1\x82\xd1\x80\xd0\xbe\xd0\xba\xd0\xb0'
```

– bytes — неизменяемая последовательность байтов. Каждый элемент последовательности может хранить целое число от о до 255, которое обозначает код символа. Объект типа bytes поддерживает большинство строковых методов и, если это возможно, выводится как последовательность символов. Однако доступ по индексу возвращает целое число, а не символ.

Пример:

```
s = bytes ("стр str", "cp1251")
print (s[0], s[5], s[0:3], s[4:7])
```

Результат:

```
(241, 116, b'\xf1\xf2\xf0', b'str')
print (s)
```

Результат:

```
b'\xf1\xf2\xf0 str'
```

Объект типа bytes может содержать как однобайтовые, так и многобайтовые символы. Обратите внимание на то, что функции и методы строк некорректно работают с многобайтовыми кодировками, — например, функция 1en() вернет количество байтов, а не символов:

```
print (len ("строка"))
```

Результат:

```
6
print (len (bytes ("строка", "cp1251")))
```

Результат:

```
6
print (len (bytes ("строка", "utf-8")))
```

Результат:

```
12
```

– bytearray — изменяемая последовательность байтов. Тип bytearray аналогичен типу bytes, но позволяет изменять элементы по индексу и содержит дополнительные методы, дающие возможность добавлять и удалять элементы.

Пример:

```
s=bytearray ("str", "cp1251")
```

```
s[0]=49
print (s)
Результат:
bytearray(b'1tr')
print (s.append(55)
Результат:
bytearray(b'1tr7')
```

Во всех случаях, когда речь идет о текстовых данных, следует использовать тип str. Именно этот тип будем называть словом «строка». Типы bytes и bytearray следует задействовать для записи бинарных данных — например, изображений, а также для промежуточного хранения текстовых данных.

4.2 Создание строки в Python

Создать строку можно следующими способами:

- **с помощью функции str ([<Объект> [, <Кодировка> [, <Обработка ошибок>]]])**. Если указан только первый параметр, то функция возвращает строковое представление любого объекта. Если параметры не указаны вообще, то возвращается пустая строка.

```
print (str(), str([1, 2]), str((3, 4)), str({'x': 1}))
Результат:
('', '[1, 2]', '(3, 4)', "{'x': 1}")
print (str(b'\xf1\xf2\xf0\xee\xea\xe0'))
Результат:
"b'\\xf1\\xf2\\xf0\\xee\\xea\\xe0'"
```

Обратите внимание на преобразование объекта типа **bytes**. Чтобы получить из объектов типа **bytes** и **bytearray** именно строку, следует указать кодировку во втором параметре:

```
print (str(b'\xf1\xf2\xf0\xee\xea\xe0', 'cp1251'))
Результат:
'строка'
```

В третьем параметре могут быть указаны значения:

- **"strict"** - при ошибке возбуждается исключение **UnicodeDecodeError** (значение по умолчанию);
- **"replace"** - неизвестный символ заменяется символом, имеющим код **\uFFFD** или
- **"ignore"** - неизвестные символы игнорируются:

```
obj1 = bytes('строка1', 'utf-8')
obj2 = bytes('строка2', 'utf-8')
print (str(obj1, 'utf-8'), str(obj2, 'utf-8'))
Результат:
('строка1', 'строка2')
print (str(obj1, 'ascii', 'strict'))
Результат:
Traceback (most recent call last):
```

```
  File "<pyshell#20>", line 1, in <module>
    str(obj1, 'ascii', 'strict')
UnicodeDecodeError: 'ascii' codec can't decode byte 0xd1 in position 0:
  ordinal not in range(128)
print (str(obj1, 'ascii', 'ignore'))
```

Результат:

```
'1'
```

– **указав строку между апострофами или двойными кавычками**:

```
print ('строка', "строка", '"x": 5', '"x': 5")
```

Результат:

```
('строка', 'строка', '"x": 5', '"x': 5")
print('Строка1\nСтрока2')
```

Результат:

```
Строка1
Строка2
print("Строка1\nСтрока2")
```

Результат:

```
Строка1
Строка2
```

В некоторых языках программирования (например, в **PHP**) строка в апострофах отличается от строки в кавычках тем, что внутри апострофов специальные символы выводятся как есть, а внутри кавычек они интерпретируются. В языке **Python** никакого отличия между строкой в апострофах и строкой в кавычках нет. Это одно и то же. Если строка содержит кавычки, то ее лучше заключить в апострофы, и наоборот. Все специальные символы в таких строках интерпретируются. Например, последовательность символов **\n** преобразуется в символ новой строки. Чтобы специальный символ выводился как есть, его необходимо экранировать с помощью слеша:

```
print("Строка1\\nСтрока2")
```

Результат:

```
Строка1\nСтрока2
print('Строка1\\nСтрока2')
```

Результат:

```
Строка1\nСтрока2
```

Кавычку внутри строки в кавычках и апостроф внутри строки в апострофах также необходимо экранировать с помощью защитного слеша:

```
print ("\"x\": 5", '\'x\': 5')
```

Результат:

```
('"x": 5', "'x': 5")
```

Следует также заметить, что заключить объект в кавычки или апострофы на нескольких строках нельзя. Переход на новую строку вызовет синтаксическую ошибку:

```
print ("string)
```

Результат:

```
SyntaxError: EOL while scanning string literal
```

Чтобы расположить объект на нескольких строках, следует перед символом перевода строки указать символ \, поместить две строки внутри скобок или использовать конкатенацию внутри скобок:

```
print ("string1\
string2") # После \ не должно быть никаких знаков
```

Результат:

```
'string1 string2'
```

```
print ("string1"
 "string2") # Неявная конкатенация строк
```

Результат:

```
'string1string2'
```

```
print ("string1" +
 "string2") # Явная конкатенация строк
```

Результат:

```
'string1string2'
```

Кроме того, если в конце строки расположен символ \, то его необходимо экранировать, иначе будет выведено сообщение об ошибке:

```
print("string\")
```

Результат:

```
SyntaxError: EOL while scanning string literal
```

```
print("string\\")
```

Результат:

```
string\
```

– **указав строку между утроенными апострофами или утроенными кавычками**. Такие объекты можно разместить на нескольких строках, допускается также одновременно использовать и кавычки, и апострофы без необходимости их экранировать. В остальном такие объекты эквивалентны строкам в апострофах и кавычках. Все специальные символы в таких строках интерпретируются. Примеры:

```
print('''Строка1
Строка2''')
```

Результат:

```
Строка1
Строка2
```

```
print("""Строка1
Строка2""")
```

Результат:

```
Строка1
Строка2
```

Если строка не присваивается переменной, то она считается строкой документирования. Такая строка сохраняется в атрибуте **__doc__** того объекта, в котором расположена. В качестве примера создадим функцию со строкой документирования, а затем выведем содержимое строки:

```
def test():
   '''Это описание функции'''
```

```
    pass
print(test.__doc__)
```

Результат:

```
Это описание функции
```

Поскольку выражения внутри таких строк не выполняются, то утроенные кавычки (или утроенные апострофы) очень часто используются для комментирования больших фрагментов кода на этапе отладки программы.

Если перед строкой разместить модификатор **r**, то специальные символы внутри строки выводятся как есть. Например, символ **\n** не будет преобразован в символ перевода строки. Иными словами, он будет считаться последовательностью двух символов: \ и **n**:

```
print("Строка1\nСтрока2")
```

Результат:

```
Строка1
Строка2
print(r"Строка1\nСтрока2")
```

Результат:

```
Строка1\nСтрока2
print(r"""Строка1\nСтрока2""")
```

Результат:

```
Строка1\nСтрока2
```

Такие неформатированные строки удобно использовать в шаблонах регулярных выражений, а также при указании пути к файлу или каталогу:

```
print(r"C:\Python34\Lib")
```

Результат:

```
C:\Python34\Lib
```

Если модификатор не указать, то все слеши при указании пути необходимо экранировать:

```
print("C:\\Python34\\Lib")
```

Результат:

```
C:\Python34\Lib
```

Если в конце неформатированной строки расположен слеш, то его необходимо экранировать. Однако следует учитывать, что этот слеш будет добавлен в исходную строку. Пример:

```
print(r"C:\Python34\Lib\")
```

Результат:

```
SyntaxError: EOL while scanning string literal
print(r"C:\Python34\Lib\\")
```

Результат:

```
C:\Python34\Lib\\
```

Чтобы избавиться от лишнего слеша, можно использовать операцию конкатенации строк, обычные строки или удалите слеш явным образом:

```
print(r"C:\Python34\Lib" + "\\") # Конкатенация
```

Результат:

```
C:\Python34\Lib\
```

```
print("C:\\Python34\\Lib\\") # Обычная строка
```
Результат:
```
C:\Python34\Lib\
print(r"C:\Python34\Lib\\"[:-1]) # Удаление слеша
```
Результат:
```
C:\Python34\Lib\
```

4.3 Специальные символы

Специальные символы — это комбинации знаков, обозначающих служебные или непечатаемые символы, которые невозможно вставить обычным способом. Приведем перечень специальных символов, допустимых внутри строки, перед которой нет модификатора r:

- \n - перевод строки;
- \r - возврат каретки;
- \t - знак табуляции;
- \v - вертикальная табуляция;
- \a - звонок;
- \b - забой;
- \f - перевод формата;
- \0 - нулевой символ (не является концом строки);
- \" - кавычка;
- \' - апостроф;
- \N - восьмеричное значение N. Например, \74 соответствует символу <;
- \xN - шестнадцатеричное значение N. Например, \x6a соответствует символу j;
- \\ - обратный слеш;
- \uxxxx - 16-битный символ Unicode. Например, \u043a соответствует русской букве к;
- \Uxxxxxxxx - 32-битный символ Unicode.

Если после слеша не стоит символ, который вместе со слешем интерпретируется как спецсимвол, то слеш сохраняется в составе строки:
```
print("Этот символ \не специальный")
```
```
Этот символ \не специальный
```
Тем не менее, лучше экранировать слеш явным образом:
```
print("Этот символ \\не специальный")
```
```
Этот символ \не специальный
```

4.4 Операции над строками

Строки относятся к последовательностям. Как и все последовательности, строки поддерживают обращение к элементу по индексу, получение среза, конкатенацию, повторение и проверку на вхождение. Рассмотрим эти операции подробно.

К любому символу строки можно обратиться как к элементу списка - достаточно указать его индекс в квадратных скобках. Нумерация начинается с нуля:

```
s = 'Python'
print(s[0], s[1], s[2], s[3], s[4], s[5])
```

Результат:

```
('P', 'y', 't', 'h', 'o', 'n')
```

Если символ, соответствующий указанному индексу, отсутствует в строке, то возбуждается исключение **IndexError**:

```
s = 'Python'
print(s[10])
```

Результат:

```
Traceback (most recent call last):
  File "<pyshell#4>", line 1, in <module>
    s[10]
IndexError: string index out of range
```

В качестве индекса можно указать отрицательное значение. В этом случае смещение будет отсчитываться от конца строки, а точнее - чтобы получить положительный индекс, значение вычитается из длины строки:

```
print(s[-1], s[len(s) - 1])
```

Результат:

```
('n', 'n')
```

Так как строки относятся к неизменяемым типам данных, то изменить символ по индексу нельзя:

```
s = 'Python'
print(s[0] = 'J')
```

Результат:

```
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    s[0] = 'J'
TypeError: 'str' object does not support item assignment
```

Чтобы выполнить изменение, можно воспользоваться операцией извлечения среза, которая возвращает указанный фрагмент строки. Формат операции:

[<Начало>:<Конец>:<Шаг>]

Все параметры здесь не являются обязательными. Если параметр **<Начало>** не указан, то используется значение 0. Если параметр **<Конец>** не указан, то возвращается фрагмент до конца строки. Следует также заметить, что символ с индексом, указанным в этом параметре, не входит в возвращаемый фрагмент. Если параметр **<Шаг>** не указан, то используется значение 1. В качестве значения параметров можно указать отрицательные значения.

Теперь рассмотрим несколько примеров. Сначала получим копию строки:

```
print( s[:] )# Фрагмент от позиции 0 до конца строки
```

Результат:

```
'Python'
```

Теперь выведем символы в обратном порядке:

```
print( s[::-1] )# Отрицательное значение третьего параметра
```

Результат:

```
'nohtyP'
```

Получим новую строку с другим первым символом:

```
print("J" + s[1:])  # Извлекаем франмент от символа 1 до конца строки
```

Результат:

```
'Jython'
```

Получим новую строку без последнего символа:

```
print(s[:-1] )# Возвращаем фрагмент от 0 до len(s) - 2
```

Результат:

```
'Pytho'
```

Напечатаем первый символ в строке:

```
print(s[0:1]) # Символ с индексом 1 не входит в диапазон
```

Результат:

```
'P'
```

А теперь получим последний символ:

```
print(s[-1:]) # Получаем фрагмент len(s) - 1 до конца строки
```

Результат:

```
'n'
```

И, наконец, выведем символы с индексами 2, 3 и 4:

```
print(s[2:5]) # Возвращаются с символы с индексами 2, 3 и 4
```

Результат:

```
'tho'
```

Узнать количество символов в строке позволяет функция **len()**:

```
print(len("Python"), len("\r\n\t"), len(r"\r\n\t")
```

Результат:

```
(6, 3, 6)
```

Теперь, когда знаем количество символов, можно перебрать все символы с помощью цикла **for**:

```
s = "Python"
for i in range(len(s)): print(s[i], end=" ")
```

Результат:

```
P y t h o n
```

Так как строки поддерживают итерации, можно просто указать строку в качестве параметра цикла. Результат выполнения будет таким же:

```
s = "Python"
for i in s: print(i, end=" ")
```

Результат:

```
P y t h o n
```

Соединить две строки в одну строку позволяет оператор +:

```
print("Строка1" + "Строка2")
```

Результат:

```
Строка1Строка2
```

Кроме того, можно выполнить неявную конкатенацию строк. В этом случае две строки указываются рядом без оператора между ними:

```
print("Строка1" "Строка2")
```

Результат:

```
Строка1Строка2
```

Обратите внимание на то, что если между строками указать запятую, то можно получить кортеж, а не строку:

```
s = "Строка1", "Строка2"
print(type(s)) # Получили кортеж, а не строку
```

Результат:

```
<class 'tuple'>
```

Если соединяются, например, переменная и строка, то следует обязательно указывать символ конкатенации строк, иначе будет выведено сообщение об ошибке:

```
s = "Строка1"
print(s + "Строка2") # Нормально
```

Результат:

```
Строка1Строка2
print(s "Строка2") # Ошибка
```

Результат:

```
SyntaxError: invalid syntax
```

При необходимости соединить строку с другим типом данных (например, с числом) следует произвести явное преобразование типов с помощью функции **str ()**:

```
print("string" + str(10))
```

Результат:

```
'string10'
```

Кроме рассмотренных операций, строки поддерживают операцию повторения, проверку на вхождение и невхождение. Повторить строку указанное количество раз можно с помощью оператора *, выполнить проверку на вхождение фрагмента в строку позволяет оператор **in**, а проверить на не вхождение – оператор **not in**:

```
print( '-' * 20)
```

Результат:

```
'--------------------'
print('yt' in 'Python') # Найдено
```

Результат:

```
True
print('yt' in 'Perl') # Не найдено
```

Результат:

```
False
print('PHP' not in 'Python') # Не найдено
```

Результат:

```
True
```

4.5 Функции и методы для работы со строками

Рассмотрим основные функции для работы со строками:

– **str([<Объект>])** - преобразует любой объект в строку. Если параметр не указан, то возвращается пустая строка. Используется функцией **print()** для вывода объектов. Пример:

```
print(str(), str([1,2]), str((3,4)), str({"x":1})
(', '[1,2]', '(3,4)', "{'x':1}")
print ("строка1\nстрока2")
```

Результат:

```
строка1
строка2
```

– **repr([<Объект>])** - возвращает строковое представление объекта. Используется при выводе объектов в окне **Python Shell** редактора **IDLE**. Пример:

```
print (repr("Строка1"), repr([1, 2, 3]), repr({"x": 5}))
```

Результат:

```
(" 'Строка' ", '[1, 2, 3]', "{'x': 5}")
print (repr("строка1\nстрока2"))
```

Результат:

```
" 'строка1\nстрока2' "
```

– **ascii([<Объект>])** - возвращает строковое представление объекта. В строке могут быть символы только из кодировки **ASCII**. Пример:

```
print (ascii([1, 2, 3]), ascii ({"x": 5}))
```

Результат:

```
('[1, 2, 3]', "{'x': 5}")
print (ascii ("строка"))
```

Результат:

```
" '\\u0441\\u0442\\u0440\\u043e\\u043a\\u0430' "
```

– **len(<Строка>)** - возвращает количество символов в строке:

```
print (len ("Python"), len ("\r\n\t"), len (r"\r\n\t"))
```

Результат:

```
(6, 3, 6)
print (len ("строка"))
```

Результат:

```
6
```

Приведем перечень основных методов для работы со строками:

– **strip([<Символы>])** - удаляет указанные в параметре символы в начале и в конце строки. Если параметр не задан, удаляются пробельные символы: пробел, символ перевода строки (**\n**), символ возврата каретки (**\r**), символы горизонтальной (**\t**) и вертикальной (**\v**) табуляции:

```
s1, s2 = "str\n\r\v\t", "strstrstrokstrstrstr"
print (" '%s' - '%s' " % (s1.strip(), s2.strip("str")))
```

Результат:

```
" 'str' - 'ok' "
```

– **lstrip([<Символы>])** - удаляет пробельные или указанные символы в начале строки:

```
s1, s2 = "   str      ", "strstrstrokstrstrstr"
print (" '%s' - '%s' " % (s1.lstrip(), s2.lstrip("str")))
```

Результат:

```
" 'str      ' - 'okstrstrstr' "
```

– **rstrip([<Символы>])** - удаляет пробельные или указанные символы в конце строки:

```
s1, s2 = "   str      ", "strstrstrokstrstrstr"
print (" '%s' - '%s' " % (s1.rstrip(), s2.rstrip("str")))
```

Результат:

```
" '         str' - 'okstrstrstr' "
```

– **split([<Разделитель>[, <Лимит>]])** - разделяет строку на подстроки по указанному разделителю и добавляет эти подстроки в список, который возвращается в качестве результата. Если первый параметр не указан или имеет значение **None**, то в качестве разделителя используется символ пробела. Во втором параметре можно задать количество подстрок в результирующем списке - если он не указан или равен -1, в список попадут все подстроки. Если подстрок больше указанного количества, то список будет содержать еще один элемент - с остатком строки. Примеры:

```
s = " word1 word2 word3"
print (s.split("\n"))
```

Результат:

```
(['word1', 'word2', 'word3'], ['word1', 'word2 word3])
s = " word1\nword2\nword3"
print (s.split("\n"))
```

Результат:

```
['word1', 'word2', 'word3']
```

Если в строке содержатся несколько пробелов подряд, и разделитель не указан, то пустые элементы не будут добавлены в список:

```
s = " word1        word2  word3       "
print (s.split())
```

Результат:

```
['word1', 'word2', 'word3']
```

При использовании другого разделителя могут возникнуть пустые элементы:

```
s = " ,,word1,,word2,,word3,,"
print (s.split(","))
```

Результат:

```
[' ', ' ', 'word1', ' ', 'word2', ' ', 'word3', ' ', ' ']
print ("1,,2,,3".split(","))
```

Результат:

```
['1', ' ', '2', ' ', '3']
```

Если разделитель не найден в строке, то список будет состоять из одного элемента, представляющего исходную строку:

```
print ("word1 word2 word3".split("\n")))
```

Результат:

```
['word1 word2 word3']
```

– **rsplit([<Разделитель>[, <Лимит>]])** - аналогичен методу **split()**, но поиск символа-разделителя производится не слева направо, а, наоборот, справа налево. Примеры:

```
s="word1 word2 word3"
print (s.rsplit(), s.rsplit(None, 1))
```

Результат:

```
(['word1', 'word2', 'word3'], ['word1 word2', 'word3'])
print ("word1 word2 word3".rsplit("\n"))
```

Результат:

```
['word1', 'word2', 'word3']
```

– **splitlines([True])** - разделяет строку на подстроки по символу перевода строки (**\n**) и добавляет их в список. Символы новой строки включаются в результат, только если необязательный параметр имеет значение **True**. Если разделитель не найден в строке, то список будет содержать только один элемент. Примеры:

```
print ("word1\nword2\nword3".splitlines())
```

Результат:

```
['word1', 'word2', 'word3']
print ("word1\nword2\nword3".splitlines(True))
```

Результат:

```
['word1\n', 'word2\n', 'word3']
print ("word1\nword2\nword3".splitlines(False))
```

Результат:

```
['word1', 'word2', 'word3']
print ("word1 word2 word3".splitlines())
```

Результат:

```
['word1 word2 word3']
```

– **partition(<Разделитель>)** - находит первое вхождение символа-разделителя в строку и возвращает кортеж из трех элементов: первый элемент будет содержать фрагмент, расположенный перед разделителем, второй элемент - сам разделитель, а третий элемент - фрагмент, расположенный после разделителя. Поиск производится слева направо. Если символ-разделитель не найден, то первый элемент кортежа будет содержать всю строку, а остальные элементы останутся пустыми. Пример:

```
print ("word1 word2 word3".partition(""))
```

Результат:

```
('word1', ' ', 'word2 word3')
print ("word1 word2 word3".partition("\n"))
```

Результат:

```
('word1 word2 word3', ' ', ' ')
```

– **rpartition(<Разделитель>)** - метод аналогичен методу **partition()**, но поиск символа-разделителя производится не слева направо, а, наоборот, справа

налево. Если символ-разделитель не найден, то первые два элемента кортежа окажутся пустыми, а третий элемент будет содержать всю строку. Пример:

```
print ("word1 word2 word3".rpartition(" "))
Результат:
('word1 word2', ' ', 'word3')
print ("word1 word2 word3".rpartition("\n"))
Результат:
(' ', ' ', 'word1 word2 word3')
```

– **join()** - преобразует последовательность в строку. Элементы добавляются через указанный разделитель. Формат метода:

<Строка> = <Разделитель>.join(<Последовательность>)

В качестве примера преобразуем список и кортеж в строку:

```
print ("=>".join(["word1", "word2", "word3"])
Результат:
'word1 => word2 => word3'
print (" ".join(("word1", "word2", "word3"))
Результат:
'word1 word2 word3'
```

Обратите внимание на то, что элементы последовательностей должны быть строками, иначе возбуждается исключение **TypeError**:

```
print (" ".join(("word1", "word2", 5))
Результат:
Traceback (most recent call last):
  File "<pyshell#23>", line 1, in <module>
    " ".join(("word1", "word2", 5))
TypeError: sequence item 2: expected str instance, int found
```

Строки относятся к неизменяемым типам данных. Если попытаться изменить символ по индексу, то возникнет ошибка. Чтобы изменить символ по индексу, можно преобразовать строку в список с помощью функции **list()**, произвести изменения, а затем с помощью метода **join()** преобразовать список обратно в строку. Пример:

```
s = "Python"
arr = list(s)
print (arr) # Преобразуем строку в список
Результат:
['P', 'y', 't', 'h', 'o', 'n']
arr[0] = "J"
print (arr) # Изменяем элемент по индексу
Результат:
['J', 'y', 't', 'h', 'o', 'n']
s = "".join(arr)
print (s) # Преобразуем список в строку
Результат:
'Jython'
```

В **Python 3** можно также преобразовать строку в тип **bytearray**, а затем изменить символ по индексу:

```
s = "Python"
b = bytearray(s, "cp1251")
print (b)
Результат:
bytearray(b'Python')
b[0] = ord("J")
print (b)
Результат:
bytearray(b'Jython')
s = b.decode("cp1251")
print (s)
Результат:
'Jython'
```

4.6 Изменение регистра символов

Для изменения регистра символов предназначены следующие методы:

- **upper()** - заменяет все символы строки соответствующими прописными буквами:

```
print("строка".upper())
Результат:
СТРОКА
```

- **lower()** - заменяет все символы строки соответствующими строчными буквами:

```
print("СТРОКА".lower())
Результат:
строка
```

- **swapcase()** - заменяет все строчные символы соответствующими прописными буквами, а все прописные символы - строчными:

```
print("СТРОКА строка".swapcase())
Результат:
строка СТРОКА
```

- **capitalize()** -делает первую букву строки прописной:

```
print("строка строка".capitalize())
Результат:
Строка строка
```

- **title()** - делает первую букву каждого слова прописной:

```
s = "первая буква каждого слова станет прописной"
print(s.title())
Результат:
Первая Буква Каждого Слова Станет Прописной
```

– **casefold()** - то же самое, что и **lower()**, но дополнительно преобразует все символы с диакрическими знаками и лигатуры в буквы стандартной латиницы. Обычно применяется для сравнения строк:

```
"Python".casefold() == "python".casefold()
Результат:
True
"grosse".casefold() == "große".casefold()
Результат:
True
```

4.7 Функции для работы с символами

Для работы с отдельными символами предназначены следующие функции:

– **chr(<*Код символа*>)** - возвращает символ по указанному коду:

```
print(chr(1055))
П
```

– **ord(<*Символ*>)** - возвращает код указанного символа:

```
print(ord('П'))
1055
```

4.8 Поиск и замена в строке

Для поиска и замены в строке используются следующие методы:

– **find()** - ищет подстроку в строке. Возвращает номер позиции, с которой начинается вхождение подстроки в строку. Если подстрока в строку не входит, то возвращается значение -1. Метод зависит от регистра символов. Формат метода:

<*Строка*>.find (<*Подстрока*> [, <*Начало*> [, <*Конец*>]])

Если начальная позиция не указана, то поиск будет осуществляться с начала строки. Если параметры **<*Начало*>** и **<*Конец*>** указаны, то производится операция извлечения среза:

<*Строка*>[<*Начало*>:<*Конец*>]

и поиск подстроки будет выполняться в этом фрагменте. Пример:

```
s = "пример пример Пример"
print(s.find("при"), s.find("При"), s.find("тест"))
(0, 14, -1)
print(s.find("при", 9), s.find("при", 0, 6), s.find("при", 7, 12))
(-1, 0, 7)
```

– **index()** - метод аналогичен методу **find()**, но если подстрока в строку не входит, то возбуждается исключение **ValueError**. Формат метода:

<*Строка*>.index(<*Подстрока*>[, <*Начало*>[, <*Конец*>]])

Пример:

```
s = "пример пример Пример"
print(s.index("при"), s.index("при", 7, 12), s.index("При", 1))
(0, 7, 14)
```

```
print(s.index("тест"))
Traceback (most recent call last):
  File "<pyshell#6>", line 1, in <module>
    s.index("тест")
ValueError: substring not found
```

– **rfind()** - ищет подстроку в строке. Возвращает позицию последнего вхождения подстроки в строку. Если подстрока в строку не входит, то возвращается значение -1. Метод зависит от регистра символов. Формат метода:

<*Строка*>.rfind(<*Подстрока*>[, <*Начало*>[, <*Конец*>]])

Если начальная позиция не указана, то поиск будет производиться с начала строки. Если параметры **<*Начало*>** и **<*Конец*>** указаны, то выполняется операция извлечения среза, и поиск подстроки будет производиться в этом фрагменте. Пример:

```
s = "пример пример Пример Пример"
print(s.rfind("при"), s.rfind("При"), s.rfind("тест"))
(7, 21, -1)
print(s.rfind("при", 0, 6), s.rfind("При", 10, 20))
(0, 14)
```

– **rindex()** - метод аналогичен методу **rfind()**, но если подстрока в строку не входит, то возбуждается исключение **ValueError**. Формат метода:

<*Строка*>.rindex(<*Подстрока*>[, <*Начало*>[, <*Конец*>]])

Пример:

```
s = "пример пример Пример Пример"
print(s.rindex("при"), s.rindex("При"), s.rindex("при", 0, 6))
(7, 21, 0)
print(s.rindex("тест"))
Traceback (most recent call last):
  File "<pyshell#14>", line 1, in <module>
    s.rindex("тест")
ValueError: substring not found
```

– **count()** - возвращает число вхождений подстроки в строку. Если подстрока в строку не входит, то возвращается значение 0. Метод зависит от регистра символов. Формат метода:

<*Строка*>.count(<*Подстрока*>[, <*Начало*>[, <*Конец*>]])

Пример:

```
s = "пример пример Пример Пример"
print(s.count("при"), s.count("при", 6), s.count("При"))
(2, 1, 2)
print(s.count("тест"))
0
```

– **startswith()** - проверяет, начинается ли строка с указанной подстроки. Если начинается, то возвращается значение **True**, в противном случае - **False**. Метод зависит от регистра символов. Формат метода:

<*Строка*>.startswith(<*Подстрока*>[, <*Начало*>[, <*Конец*>]])

Если начальная позиция не указана, сравнение будет производиться с началом строки. Если параметры **<*Начало*>** и **<*Конец*>** указаны, то выполняется операция извлечения среза, и сравнение будет производиться с началом фрагмента. Пример:

```
s = "пример пример Пример Пример"
print(s.startswith("при"), s.startswith("При"))
(True, False)
print(s.startswith("при", 6), s.startswith("При", 14))
(False, True)
```

– **endswith()** - проверяет, заканчивается ли строка указанной подстрокой. Если заканчивается, то возвращается значение **True**, в противном случае - **False**. Метод зависит от регистра символов. Формат метода:

<*Строка*>.endswith(<*Подстрока*>[, <*Начало*>[, <*Конец*>]])

Если начальная позиция не указана, то сравнение будет производиться с концом строки. Если параметры **<*Начало*>** и **<*Конец*>** указаны указаны, то выполняется операция извлечения среза, и сравнение будет производиться с концом фрагмента. Пример:

```
s = "подстрока ПОДСТРОКА"
print(s.endswith("ока"), s.endswith("ОКА"))
(False, True)
print(s.endswith("ока", 0, 9))
True
```

– **replace()** - производит замену всех вхождений заданной подстроки в строке на другую подстроку и возвращает результат в виде новой строки. Метод зависит от регистра символов. Формат метода:

<*Строка*>.replace(<*Подстрока для замены*>, <*Новая подстрока*> [, <*Максимальное количество замен*>])

Если количество замен не указано, будет выполнена замена всех найденных подстрок. Пример:

```
s = "Привет, Петя"
print(s.replace("Петя", "Вася"))
Привет, Вася
print(s.replace("петя", "вася")) # Зависит от регистра
Привет, Петя
s = "strstrstrstrstrstr"
print(s.replace("str",""), s.replace("str","", 3))
('', 'strstrstr')
```

– **translate(<*Таблица символов*>)** - заменяет символы в соответствии с параметром **<*Таблица символов*>**. Параметр **<*Таблица символов*>** должен быть словарем, ключами которого являются **Unicode**-коды заменяемых символов, а значениями - **Unicode**-коды заменяющих символов. Если в качестве значения указать **None**, то символ будет удален. Для примера удалим букву ***П***, а также изменим регистр всех букв ***р***:

```
s = "Пример"
d = {ord("П"): None, ord("р"): ord("Р")}
```

```
print(d)
{1055: None, 1088: 1056}
print(s.translate(d))
'РимеР'
```

Упростить создание параметра <***Таблица символов***> позволяет статический метод **maketrans ()**. Формат метода:

```
str.maketrans(<X>[, <Y>[, <Z>]])
```

Если указан только первый параметр, то он должен быть словарем:

```
t = str.maketrans({"a": "A", "o": "O", "c": None})
print(t)
{1072: 'A', 1086: 'O', 1089: None}
print("строка".translate(t))
'трОкА'
```

Если указаны два первых параметра, то они должны быть строками одинаковой длины. В результате будет создан словарь с ключами из строки **<X>** и значениями из строки **<Y>**, расположенными в той же позиции. Изменим регистр некоторых символов:

```
t = str.maketrans("абвгдежзи", "АБВГДЕЖЗИ")
print(t)
{1072: 1040, 1073: 1041, 1074: 1042, 1075: 1043, 1076: 1044, 1077: 1045,
1078: 1046, 1079: 1047, 1080: 1048}
print("абвгдежзи".translate(t))
'АБВГДЕЖЗИ'
```

В третьем параметре можно дополнительно указать строку из символов, которым будет сопоставлено значение **None**. После выполнения метода **translate()** эти символы будут удалены из строки. Заменим все цифры на 0, а некоторые буквы удалим из строки:

```
t = str.maketrans("123456789", "0" * 9, "str")
print(t)
{49: 48, 50: 48, 51: 48, 52: 48, 53: 48, 54: 48, 55: 48, 56: 48, 57: 48, 115: None,
116: None, 114: None}
print("str123456789str".translate(t))
'000000000'
```

4.9 Пример решения задачи на языке программирования Python

Задача. Пользователь вводит строку row, содержащую несколько слов, между которыми один или несколько пробелов. Требуется найти количество символов в самом длинном слове.

Комментарий. В цикле с оператором **for** мы проверяем наличие пробела в строке, указывающее на переход к очередному слову. В переменной **kol** будет храниться количество символов в слове. Соответственно, для того чтобы найти максимальную длину слова, мы вводим в программу такую переменную, как **maxim,** предварительно присвоив ей маленькое значение (например, –10).

Таким образом, если значение ячейки **kol** превышает значение ячейки **maxim,** то находится самое длинное слово в строке при текущем проходе цикла. Следовательно, на выходе из цикла в ячейке **maxim** будет храниться количество символов в самом длинном слове строки.

Решение задачи:

```
stroka=input("Введите строку\n ")
kol=0
maxim=-10
for i in range(len(stroka)):
    if stroka[i]!=" ":
        kol=kol+1
        if kol>maxim:
            maxim=kol
    else:
        kol=0
print("\nВ самом длинном слове ", maxim, "символов")
```

5 Индивидуальное задание

5.1 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Введите вашу фамилию, имя и отчество как строку символов. Определите длину строки и количество букв, совпадающих со второй буквой фамилии. Выведите имя и количество букв в третьем слове.

2 Введите вашу фамилию, имя и отчество как строку символов. Определите длину строки и количество букв, совпадающих со второй буквой фамилии. Определите количество букв «а» в фамилии.

3 Введите вашу фамилию, имя и отчество как строку символов. Определите длину строки и количество букв, совпадающих со второй буквой фамилии. Выведите первые буквы фамилии, имени и отчества с точками.

4 Введите вашу фамилию, имя и отчество как строку символов. Определите длину строки и количество букв, совпадающих со второй буквой фамилии. Выведите длину фамилии и имени.

5 Введите вашу фамилию, имя и отчество как строку символов. Определите длину строки и количество букв, совпадающих со второй буквой фамилии. Выведите фамилию и инициалы.

6 Введите вашу фамилию, имя и отчество как строку символов. Определите длину строки и количество букв, совпадающих со второй буквой фамилии. Выведите имя и количество букв в фамилии.

7 Введите вашу фамилию, имя и отчество как строку символов. Определите длину строки и количество букв, совпадающих со второй буквой фамилии. Определите количество букв «а» в имени.

8 Введите вашу фамилию, имя и отчество как строку символов. Определите длину строки и количество букв, совпадающих со второй буквой фамилии. Выведите самое длинное слово.

9 Введите вашу фамилию, имя и отчество как строку символов. Определите длину строки и количество букв, совпадающих со второй буквой фамилии. Удалите все буквы «а» и «о» из фамилии.

10 Введите вашу фамилию, имя и отчество как строку символов. Определите длину строки и количество букв, совпадающих со второй буквой фамилии. Выведите имя в столбец.

11 Введите вашу фамилию, имя и отчество как строку символов. Определите длину строки и количество букв, совпадающих со второй буквой фамилии. Выведите имя, фамилию и суммарную длину слов.

12 Введите вашу фамилию, имя и отчество как строку символов. Определите длину строки и количество букв, совпадающих со второй буквой фамилии. Каждую букву имени продублируйте.

13 Введите вашу фамилию, имя и отчество как строку символов. Определите длину строки и количество букв, совпадающих со второй буквой фамилии. Выведите фамилию в обратном порядке.

14 Введите вашу фамилию, имя и отчество как строку символов. Определите длину строки и количество букв, совпадающих со второй буквой фамилии. Подсчитайте количество гласных букв во всей строке.

15 Введите вашу фамилию, имя и отчество как строку символов. Определите длину строки и количество букв, совпадающих со второй буквой фамилии. Замените все пробелы символом «-» и выведите полученную строку.

5.2 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 Дана строка символов, состоящая из произвольного текста на английском языке, слова разделены пробелами. В каждом слове замените первую букву на прописную.

2 Дана строка символов, состоящая из произвольного текста на английском языке, слова разделены пробелами. Удалите первую букву в каждом слове.

3 Дана строка символов, состоящая из произвольного текста на английском языке, слова разделены пробелами. Замените местами первую и последнюю буквы каждого слова.

4 Сформирйте новую строку, поменяв местами первое и последнее слово из исходной строки.

5 Дана строка, состоящая из групп нулей и единиц. Посчитайте количество нулей и единиц.

6 Посчитайте в строке количество слов.

7 Найдите количество знаков препинания в исходной строке.

8 Дана строка символов. Выведите на экран цифры, содержащиеся в строке.

9 Дана строка символов, состоящая из произвольных десятичных цифр, разделенных пробелами. Выведите количество четных чисел в этой строке.

10 Дана строка символов. Выведите на экран количество строчных русских букв, входящих в эту строку.

11 Дана строка символов. Выведите на экран только строчные русские буквы, входящие в эту строку.

12 Дана строка символов, состоящая из произвольного текста на английском языке, слова разделены пробелами. В каждом слове замените первую букву на прописную.

13 Дана строка символов, состоящая из произвольного текста на английском языке, слова разделены пробелами. Удалите первую букву в каждом слове.

14 Дана строка символов, состоящая из произвольного текста на английском языке, слова разделены пробелами. В каждом слове удалите последнюю букву.

15 Дана строка символов. Подсчитайте количество гласных и согласных букв, содержащихся в строке.

6 Содержание отчета (отчет по лабораторной работе выполняется в электронном виде в папке учащегося)

6.1 Тема лабораторной работы
6.2 Цель лабораторной работы
6.3 Индивидуальное задание (текст программы и скриншоты выполнения)

7 Контрольные вопросы

7.1 Опишите операции над строками в языке программирования Python.

7.2 Приведите синтаксис и опишите способы создания строк в языке программирования Python.

7.3 Охарактеризуйте функции для работы с символами в языке программирования Python.

7.4 Опишите функции и методы для работы со строками в языке программирования Python.

Список используемых источников

1 Постолит, А. В. Python, Django и PyCharm для начинающих / А. В. Постолит. – СПб.: БХВ-Петербург, 2021. – 464 с.: ил.

2 Прохоренок, Н. А. Python 3. Самое необходимое / Н. А. Прохоренок, В. А. Дронов. – 2-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2019. – 608 с.: ил.

3 Фоминых, Е. И. Инструментальное программное обеспечение: учебное пособие / Е. И. Фоминых, Т. Е. Фоминых. – Минск : РИПО, 2022. – 410 с.: ил.